

T1 — PISMO-4412

1. Verification Design

The defect is proven or disproven by sending N concurrent POST /transactions requests with the same idempotency key and the same request body, then querying a state/count oracle. The assertion is count == 1. Response replay equality is not a sufficient oracle.

1. Create an isolated account for the test.
2. Generate a unique idempotency key for the test.
3. Create a transaction body: account_id, amount and operation_type_id.
4. Start N concurrent POST /transactions requests using the same key and the same body.
5. Collect all responses for diagnostics. Do not use response equality as the state oracle.
6. Query the transaction-state/count oracle for the created account and assert exactly one transaction exists.
7. Record returned transaction_id(s), statuses and oracle count as failure evidence.

2. Executable Pytest Test:

https://github.com/AMANSINGHC/pismo_automation-framework/blob/c284ca365bde7bf90cf0e58e4685d4e336209a3/tests/transactions/test_create_transaction.py#L129

Note: transaction-state/count oracle cannot be implemented using the documented endpoints. Assertion on uniqueness of transaction_id(s) received from “POST /transactions” endpoint is used as diagnostic evidence. The release-blocking assertion remains transaction_record_count == 1.

3. The Observability Platform

The published APIs in the contract cannot prove “exactly one transaction”. No transaction state endpoint is exposed.

- **What can be verified today:** request/response schema, documented status/media type behavior, and execution of concurrent requests.
- **What cannot be verified today:** the number of transaction records created by the concurrent requests.
- **Why this blocks sign-off:** duplicate money movement is a P0 risk. Without a transaction state oracle, a green replay test cannot distinguish one transaction write from multiple writes that happen to return similar responses.
- **Minimum contract/API change:** expose a transaction-state/count query/API that can identify transactions for an account.

- The contract must also explicitly define Idempotency-Key presence, format, same-key/same-body replay, same-key/different-body conflict, and missing-key behavior.

4. Root Cause Hypothesis

Rank	Hypothesis	What happens?	How to detect it?
1	Check-then-insert race condition	Two requests check whether the same idempotency key exists at the same time. Both see that it doesn't exist, so both create the transaction and idempotency record.	Logs show both requests checked the key before either one was saved. Duplicate transactions then appear with the same idempotency key in the DB.
2	Incorrect key fingerprint handling	The same idempotency key and request body are sent again, but the system does not recognize it as a duplicate request.	Send the exact same request multiple times with the same idempotency key. If multiple transactions are created or the responses are inconsistent instead of returning the original result, idempotency handling is incorrect.

5. Regression Guardrail

The exact regression guardrail is the concurrent same-key test above, executed against a real provider in the pre-release gate once the transaction-state/count oracle exists. It should also run nightly against staging to detect regressions earlier.

Avoid flaky test:

1. use isolated accounts, unique idempotency key, and fixed request body per test.
2. never auto-retry the test after an assertion failure.

6. Production Metric/Alert

Signal	Source	Threshold	Who gets paged?
Duplicate transaction rate	Compare transaction records and idempotency records using the idempotency key and transaction ID to identify duplicate transactions.	The duplicate rate should be 0. Any confirmed duplicate transaction for the same idempotency key should trigger a P0 alert.	Payments Operations and the service on-call
Audit mismatch	Compare transaction	There should be no	Payments

rate	records with the audit records to find differences in transaction data.	unexplained mismatches. Any mismatch should trigger an immediate alert.	Operations and the service on-call
POST /transactions retry rate	POST /transactions metrics	Alert when retry rate is significantly above the normal baseline	Service on-call

7. Systematic Fix

7.1 Code Change

- Use a database uniqueness constraint or similar mechanism so that concurrent requests cannot both use the same idempotency key.
- Bind the idempotency key to the request body:
 - Same key + same request → return the original result.
 - Same key + different request → return a consistent conflict error.
- When the idempotency record and the transaction are saved and committed, they should be part of an atomic transaction so that they cannot get out of sync.
- Add logging for the request ID, idempotency key, transaction ID, and audit log so duplicates can be traced across services without depending on customer reports.

7.2 Contract Change

- Document idempotency key as required, including format.
- Document replay rules:
 - Same key + same request → return the original result.
 - Same key + different request → return a consistent conflict error.
- Document missing key behavior.
- Provide a reliable way to check the transaction state or count to confirm that exactly one transaction was created.

7.3 CI Gates

- **PR:** Contract validation; Contract tests; Critical account, transaction and HTTP tests.
- **Nightly:** Full boundaries, money probe, idempotency scenarios, very large inputs and additional negative scenarios.
- **Prerelease:** Full critical suite, idempotency scenarios, transaction state and audit integration checks.

7.4 Review Process

- Require explicit QA review for changes to idempotency, database constraints, transaction boundaries, or money movement.
- Require consumer-contract verification for the mobile app and downstream ledger service.